Cloud Drive

A full-stack project built for LUT University's Advanced Web Applications course

GitHub Repository: https://github.com/aahmad-1/Cloud-Drive

Project Overview

This project develops a full-stack document storage application that lets users register, log in, and create/edit/organize their own documents in a drive. The concept is similar to that of Google Drive or any similar platform. The application supports text editing in an editor, image uploads, sharing documents with other users, downloading documents, a document recycle bin, an editing lock system that prevents simultaneous edits, and more.

Tech Stack

Frontend: TypeScript, Vite, React, React Router, Bootstrap, Quill, i18next, jsPDF, react-icons

Backend: TypeScript, Node.js, Express, express-validator, MongoDB, JWT, bcrypt, Multer

Table of Contents

Application Features

Authentication & Security

- JWT-based registration and login with hashed passwords
- Editing lock system to prevent simultaneous conflicting edits

Document Management

- Create, edit, rename, and delete text and image documents
- Text editing with Quill
- Clone existing documents
- Recycle bin with restore and permanent delete

Collaboration & Sharing

- Grant or revoke edit access to specific users
- Share documents via public, read-only links

Organization & Discovery

- Search, sort, and paginated document listing
- Creation and last-updated timestamps

Export & Media

- Download documents/images as PDF
- Direct image download
- Profile picture upload with default letter-avatar fallback

User Experience

- Full English/Finnish translation
- Dark and light mode
- Responsive design across desktop and mobile

Installation Guide

Prerequisites:

- Node.js
- MongoDB running locally (downloading *Compass* is suggested)
- A web browser

Steps:

1. Clone the repository

➢ *git clone* https://github.com/aahmad-1/Cloud-Drive.git (skip if already downloaded)
➢ cd Cloud-Drive

2. Install dependencies

➢ npm install
➢ cd client && npm install (check *Miscellaneous Notes* if these don't work)
➢ cd ../server && npm install

3. Configure environment variables

➢ Create an *.env* file inside the *server* folder and add the following:
  - SECRET=secret_key
  - PORT=3000
  - MONGO_URI=mongodb://127.0.0.1:27017/clouddrivedb

4. Run the application

➢ Make sure you add a connection to *mongodb://localhost:27017* on Compass
➢ Make sure you're in the project root and run *npm run dev*
➢ Open http://localhost:5173/ on your browser

This starts the backend on http://localhost:3000 and the frontend on http://localhost:5173 at the same time. See *requirements.txt* in the repository for the exact commands used to set up the project and install every dependency from scratch.

User Manual

1. **Register/Login**: First, create an account with a username (3-25 characters) and a password (minimum of 8 characters, with at least one lowercase letter, one uppercase letter, one number, and one symbol). Log in with the same credentials after registering.
2. **Messages**: Success and error messages appear throughout the app to help users. Mainly on error pages and after actions like share, revoke, login, and register.
3. **Drive**: After logging in, you land on your Drive.
    - You can create a new text document by clicking the *Create* button. You can upload an image by choosing a file and then clicking the *Upload Image* button (titles are defaulted to the original image file name).
    - You can search your documents, sort them, and page through them with the *Show More* button at the bottom.
4. **Navigation Bar**: The top left features a button to return to the drive if you're in the recycle bin or profile page. The top right features buttons to toggle between light and dark mode, to go to your profile, to log out, and to switch the UI's language (EN & FI).
5. **Profile**: Upload a profile picture from the Profile page. If none is set or the uploaded file is not in an image format, a default letter avatar (based on the username) is given instead.
6. **Creating & Editing Documents**: Click a document's title to open it. Text documents feature a text editor while image documents are shown directly. Remember to click the *Save* button after editing the text contents or title!
7. **Sharing**: On the owner's view of a document, you can give edit access to another user by username, or toggle a public link that lets anyone view the document (read-only) without having to log in. Owners can revoke edit access by username as well.
8. **Downloading**: Both text documents and image documents can be downloaded as a PDF. Text documents downloaded as a PDF lose any formatting done in the text editor, while images downloaded as a PDF take up the whole PDF page. When image documents that can move (GIF, webp, etc) are downloaded as a PDF, only the first frame is downloaded. Images can also be downloaded directly in their original format.
9. **Recycle bin**: Deleting a document first moves it to the recycle bin (accessible via the trash icon on the Drive page), from which it can be restored or permanently deleted.

API Endpoints

Auth

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Log in, returns a JWT
GET	/api/auth/me	Get the logged-in user's info
PUT	/api/auth/profile-picture	Upload a profile picture

Documents

Method	Endpoint	Description
GET	/api/documents	Get all documents owned by or shared with the user
POST	/api/documents	Create a new document
GET	/api/documents/trash	Get all documents in the recycle bin
GET	/api/documents/:id	Get a single document (owner, editor, or public view)
PUT	/api/documents/:id	Edit a document's title and/or content
DELETE	/api/documents/:id	Move a document to the recycle bin
PUT	/api/documents/:id/share	Give edit access to a user by their username
PUT	/api/documents/:id/revoke	Revoke edit access from a user by their username
PUT	/api/documents/:id/public	Toggle the public read-only link
PUT	/api/documents/:id/restore	Restore a document from the recycle bin
DELETE	/api/documents/:id/permanent	Permanently delete a document
POST	/api/documents/:id/clone	Clone an existing document
POST	/api/documents/upload-image	Upload an image as a new document
PUT	/api/documents/:id/lock	Claim the editing lock on a document
PUT	/api/documents/:id/unlock	Release the editing lock on a document

Known Limitations

- **Responsive testing:** I tested the application on various screen sizes on Chrome DevTools' Device Mode to get a clearer idea of how it would look and work on desktop and mobile environments. The smallest width I supported for was 375px. This is because it was the smallest preset device width in Device Mode on Chrome DevTools that I could find. Beyond this, the application should display correctly across most screen sizes.
- **File Uplload Validation:** No file validation was implemented for the process of uploading profile pictures. Uploading a non-image file causes it to revert to the default letter avatar rather than displaying an error. For image documents in the drive, uploading an unsupported file type (for example a PDF) shows a placeholder graphic with a message below it which informs the user the uploaded image isn't a supported file type and also lists various supported image formats.
- **PDF Formatting**: Any formatting applied in the document editor (like bolding, underlining, italicizing, bullet points, etc.) will not translate over when a text document is downloaded as a PDF.
- **Animated Images:** Moving images (like GIFs and animated WebPs) downloaded from the drive will still have their original animation, since the download is just a direct copy of the original uploaded file. A downloaded file's animation depends on which app opens it. Windows built-in *Photos* app, for example, shows the animation in GIF files but doesn't for animated WebP files. However, opening either file in a browser animates both correctly. So if a downloaded image looks still, try to open it in a browser before assuming something is broken. Additionally, downloading an animated image as a PDF only includes its first frame, since PDFs cannot contain animated content.
- **Editing Lock:** The editing lock is tied to the browser tab being open. If a user's device loses power or crashes without closing the tab normally, the lock could remain until manually cleared by the user who was editing it last (since beforeunload/sendBeacon can't guarantee delivery in every possible scenario).

Miscellaneous Notes

- The commands in step 2 of the installation guide may not work on older versions of PowerShell. Make sure you either update PowerShell or switch the terminal to *Command Prompt* or *Git Bash,* as they both already support && commands natively.
- Visiting a page that requires authentication (Drive, Trash, Profile) while logged out shows an informational message with a guide on how to navigate back to the login page, rather than redirecting automatically.
- Uploaded images (profile pictures and image documents) are stored on the server's file system, with their file path stored in MongoDB.
- Date and time displayed in tooltips follow the currently selected UI language (English or Finnish), and not the visitor's device settings like *Created* and *Last Updated* do.
- The dark mode settings reset to light mode when the tab is closed (uses sessionStorage).

Acknowledgements

- Light & dark mode and recycle bin icons taken from react-icons
- Document text editor made possible by Quill
- Document → PDF converter made possible by jsPDF
- Responsive design + UI components from Bootstrap
- Internationalization powered by i18next and react-i18next